CCPS LAB EXAM

Last-Minute Practical Cheatsheet

Socket Programming • GPIO Interfacing • Node-RED • MQTT • Dashboard

How to use this sheet: Memorize the flow/order first, then practise the short code. These topics are based primarily on the CCPS lab records you uploaded. Hardware pins and URLs are noted where relevant.

Exam workflow: Aim → connections/configuration → code/flow → run/deploy → verify output → result.

Contents

• 1. Socket programming (TCP client/server)

• 2. Ultrasonic sensor + LED + socket

• 3. Raspberry Pi GPIO LED interfacing

• 4. Arduino LED basics

• 5. Node-RED: editor, flow and dashboard

• 6. Node-RED Function node patterns

• 7. MQTT in Node-RED

• 8. Light intensity monitoring flow

• 9. Smart lab monitoring logic

• 10. Common viva questions and troubleshooting

1. Socket Programming — TCP Client/Server

Idea: The client connects to the server over Wi-Fi and sends data. The server waits for the connection and prints received data. Your record uses TCP, port 5683.

Server code (server.py)

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("0.0.0.0", 5683))
server.listen(1)
print("Waiting for client...")

conn, addr = server.accept()
print("Client connected:", addr)

try:
    while True:
        data = conn.recv(1024).decode("utf-8")
        if not data:
            break
        print("Received:", data)
except KeyboardInterrupt:
    print("Server stopped")
finally:
    conn.close()
    server.close()
```

Client code (client.py) — simulated sensor value

```
import socket
import time

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("192.168.1.10", 5683))  # replace with server IP

try:
    while True:
        distance = 15
        message = f"distance:{distance}"
        client.sendall(message.encode("utf-8"))
        print("Sent:", message)
        time.sleep(2)
except KeyboardInterrupt:
    print("Client stopped")
finally:
    client.close()
```

Run order: Server: `python3 server.py` first. Client: `python3 client.py` second. Both devices must be on the same Wi-Fi; the client uses the server's actual IP.

Code / term	Meaning
AF_INET	IPv4 address family
SOCK_STREAM	TCP socket
bind()	Attach server to IP/interface and port
listen(1)	Wait for client connection
accept()	Accept a client
recv(1024)	Receive up to 1024 bytes
encode() / decode()	Text ↔ bytes
sendall()	Send message bytes

2. Ultrasonic Sensor + LED + Socket

Based on your Experiment 8B. Raspberry Pi BCM pins: TRIG 18, ECHO 24, LED 17. The sensor-reading part runs on the client; the client sends each reading to the server.

Ultrasonic reading function (Raspberry Pi)

```
import RPi.GPIO as GPIO
import time

TRIG = 18
ECHO = 24
LED = 17
THRESHOLD = 20

GPIO.setmode(GPIO.BCM)
GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)
GPIO.setup(LED, GPIO.OUT)

def read_distance():
    GPIO.output(TRIG, False)
    time.sleep(0.5)
    GPIO.output(TRIG, True)
    time.sleep(0.00001)  # 10 microseconds
    GPIO.output(TRIG, False)

    start = time.time()
    while GPIO.input(ECHO) == 0:
        start = time.time()

    end = time.time()
    while GPIO.input(ECHO) == 1:
        end = time.time()

    duration = end - start
    distance = round(duration * 17150, 2)
    return distance
```

Use inside the client loop:

```
dist = read_distance()
print("Distance:", dist, "cm")

if dist <= THRESHOLD:
    GPIO.output(LED, GPIO.HIGH)
else:
    GPIO.output(LED, GPIO.LOW)

client_socket.sendall(f"distance:{dist:.2f}".encode())
time.sleep(2)
```

Formula: distance (cm) ≈ echo pulse duration (seconds) × 17150. The record uses a 20 cm threshold to switch the LED.

Safety: Raspberry Pi GPIO is 3.3 V. Many HC-SR04 echo pins output 5 V; use a suitable voltage divider/level shifter before connecting ECHO to the Pi.

3. Raspberry Pi GPIO — LED Blink

Wiring from your lab record: LED long leg (anode) → GPIO 17 (physical pin 11); short leg (cathode) → 330 Ω resistor → GND.

```
import RPi.GPIO as GPIO
import time

LED_PIN = 17
GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(LED_PIN, GPIO.OUT)

try:
    while True:
        GPIO.output(LED_PIN, GPIO.HIGH)  # ON
        time.sleep(1)
        GPIO.output(LED_PIN, GPIO.LOW)   # OFF
        time.sleep(1)
except KeyboardInterrupt:
    print("Exiting...")
finally:
    GPIO.cleanup()
```

Run: save as `led_control.py`, then run `python3 led_control.py`. Stop with Ctrl+C.

Function	Use
GPIO.setmode(GPIO.BCM)	Use BCM GPIO numbers
GPIO.setup(pin, GPIO.OUT)	Configure output pin
GPIO.setup(pin, GPIO.IN)	Configure input pin
GPIO.output(pin, GPIO.HIGH)	Output HIGH / turn LED on
GPIO.output(pin, GPIO.LOW)	Output LOW / turn LED off
GPIO.input(pin)	Read digital input
GPIO.cleanup()	Reset GPIO settings on exit

4. Arduino — LED Blink Basics

```
int led = 8;

void setup() {
  pinMode(led, OUTPUT);
}

void loop() {
  digitalWrite(led, HIGH);
  delay(1000);
  digitalWrite(led, LOW);
  delay(1000);
}
```

Remember: Arduino uses `setup()` once and repeats `loop()`. `pinMode()` configures a pin; `digitalWrite()` sets output; `delay(1000)` waits 1 second.

5. Node-RED — Open, Build, Deploy

How to open Node-RED

1. Start Node-RED from a terminal with `node-red` (if installed).
2. Open the editor: **http://localhost:1880**
3. Drag nodes into the workspace, configure them and connect the grey ports with wires.
4. Click **Deploy**.
5. Click the Inject button to trigger the flow.

How to open the classic dashboard

Classic Dashboard URL: **http://localhost:1880/ui**. Install `node-red-dashboard` from Menu → Manage palette → Install if the Gauge/UI nodes are missing. Configure the UI node with a Dashboard Tab and Group, then Deploy.

Hello World flow

Flow: **Inject** → **Debug**. Set Inject payload type to string and value to `Hello World`; connect to Debug; Deploy; click Inject; view the Debug sidebar.

Display current time

```
msg.payload = new Date().toString();
return msg;
```

Flow: Inject → Function → Debug. This is the exact simple pattern from your Lab 1 record.

6. Node-RED — Dashboard and Function Patterns

A. Temperature gauge

Flow: Inject (number) → Gauge. Set Inject to a number such as 28. Configure Gauge label = Temperature, units = °C, min = 0, max = 100, and choose a Tab/Group. Deploy and open `http://localhost:1880/ui`.

B. High temperature alert

```
if (msg.payload > 40) {
    msg.payload = "High Temperature Alert!";
} else {
    msg.payload = "Temperature Normal";
}
return msg;
```

Flow: Inject → Function → Debug (or Text widget). Inject a numeric temperature. The example uses a threshold of 40.

C. Distance classification

```
let distance = Number(msg.payload);

if (distance > 50) {
    msg.payload = "SAFE";
} else if (distance >= 20) {
    msg.payload = "WARNING";
} else {
    msg.payload = "DANGER";
}
return msg;
```

D. Dashboard output rules

Widget	Typical payload
Gauge	Number
Chart	Number or time-series value
Text / Debug	Text or number
Switch / Button	Usually sends a configured value

Important: A Gauge needs a numeric payload. If a Function changes the payload to words such as 'HIGH TEMPERATURE', connect it to a Text/Debug node instead, or create a separate output path for the numeric gauge.

7. MQTT in Node-RED

MQTT: lightweight publish/subscribe messaging. A publisher sends to a topic; a subscriber receives messages from that topic through a broker. Your lab examples use port 1883 for a local broker.

A. Light ON/OFF

Flow: Inject ON + Inject OFF → MQTT Out → broker/topic. Add MQTT In with the same broker and topic → Debug. Set payloads to strings `ON` and `OFF`. Deploy.

B. High temperature alert over MQTT

Flow: Inject → Function → MQTT Out (topic e.g. `temperature/alert`). MQTT In subscribes to the same topic → Debug.

```
if (msg.payload > 40) {
    msg.payload = "High Temperature Alert!";
} else {
    msg.payload = "Temperature Normal";
}
return msg;
```

C. Multiply two numbers using Join

Flow: two Inject nodes → Join (manual mode; create array) → Function → MQTT Out. Subscriber: MQTT In → Debug. Set the two Inject payloads to numbers.

```
var num1 = msg.payload[0];
var num2 = msg.payload[1];
msg.payload = num1 * num2;
return msg;
```

MQTT setup checklist: Publisher and subscriber use the same broker; subscriber listens to the same topic that publisher sends to; confirm the broker is running; Deploy.

8. Light Intensity Monitoring — Raspberry Pi + Node-RED

Your uploaded Experiment 8 uses an LDR, Raspberry Pi, TCP socket and Node-RED Gauge. The Raspberry Pi sends a scaled light reading; Node-RED receives it through TCP-in and displays it in a Gauge. The lab record uses server port 5000.

Node-RED flow idea

TCP in → **Change/Function (convert payload to number)** → **Gauge**. Configure TCP-in to connect to the server/port used by your experiment; use the actual Pi IP address and port 5000. If your flow uses MQTT too, connect TCP-in → MQTT Out and subscribe with MQTT In.

Server-side core pattern from the record

```
import socket
import time
import RPi.GPIO as GPIO

LDR_PIN = 18
SERVER_IP = "0.0.0.0"
SERVER_PORT = 5000

GPIO.setmode(GPIO.BCM)

# rc_time(pin) measures a relative charge time
# using the circuit in the lab record.

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((SERVER_IP, SERVER_PORT))
server.listen(1)
print("Waiting for client...")
conn, addr = server.accept()

try:
    while True:
        light_value = rc_time(LDR_PIN)
        scaled = max(0, min(100, int(100 - light_value / 1000)))
        conn.sendall((str(scaled) + "\\n").encode())
        time.sleep(2)
finally:
    conn.close()
    server.close()
    GPIO.cleanup()
```

Note: The full `rc_time(pin)` function and circuit are required for actual hardware; this short block shows the server/data-sending structure. Ensure Node-RED's TCP-in mode and delimiter match the sender's newline-delimited readings.

9. Smart Laboratory Monitoring — Decision Logic

Your Smart Laboratory Monitoring record uses an object with numeric temperature/humidity, Boolean motion, and optional distance. Example input: `{"temperature":35,"humidity":60,"motion":true,"distance":15}`.

```
let d = msg.payload;
let temp = d.temperature;
let humidity = d.humidity;
let motion = d.motion;
let distance = Number(d.distance ?? 100);

let status;
if (temp > 30 && motion) {
    status = "CRITICAL CONDITION";
} else if (temp > 30 && !motion) {
    status = "HIGH TEMPERATURE";
} else if (temp <= 30 && motion) {
    status = "PERSON DETECTED";
} else {
    status = "LAB NORMAL";
}

let led = status === "CRITICAL CONDITION" ? "ON" : "OFF";
msg.payload = {
    temperature: temp,
    humidity: humidity,
    motion: motion ? "MOTION DETECTED" : "NO MOTION",
    status: status,
    led: led,
    distance: distance
};
return msg;
```

Condition	Status
temp > 30 and motion true	CRITICAL CONDITION
temp > 30 and motion false	HIGH TEMPERATURE
temp ≤ 30 and motion true	PERSON DETECTED
temp ≤ 30 and motion false	LAB NORMAL

The Function node expects a JavaScript object, not a plain text string. Use a Debug node to inspect the output object; use a JSON node or suitable Change/Function logic if incoming data arrives as JSON text.

10. Viva Questions + Troubleshooting

Question	Short answer
What is a socket?	An endpoint for communication between processes/devices.
TCP vs UDP?	TCP is connection-oriented and reliable; UDP is connectionless and does not guarantee delivery.
What is a port?	A numbered communication endpoint on a device.
What is MQTT?	A lightweight publish/subscribe messaging protocol.
What is a broker?	The MQTT server that routes published messages to subscribers.
What does Inject do?	Starts a flow by sending a configured message.
What does Debug do?	Displays messages in Node-RED's Debug sidebar.
What does Deploy do?	Applies the flow changes and activates the flow.
Why encode/decode?	Socket APIs send bytes; encode text to bytes and decode received bytes to text.
Why GPIO.cleanup()?	Releases/reset GPIO pin settings after the program exits.

Common issues

• **Connection refused:** start server first; check IP/port and firewall.
• **Address already in use:** stop the previous server process or use another port.
• **No dashboard:** install the right dashboard package, configure Tab/Group, Deploy, open the correct URL.
• **Gauge blank/wrong:** it expects a number; check Inject type and Function output.
• **MQTT not connected:** check broker service, host/port, topic and matching publisher/subscriber settings.
• **LED not blinking:** check BCM vs physical numbering, GND, resistor, wiring, and run on the Raspberry Pi.

Final practical checklist: Know the wiring → write/save code → run or Deploy → trigger input → verify output → explain the result. Use the pin numbers and ports specified by the exact experiment question.